

13-Python Loops Practice Class

Introduction

- The class focuses on practicing Python loop constructs and handling related coding exercises.
- The session's main goals are deepening understanding of loops, practicing problem-solving through code, and preparing for upcoming topics such as lists.

Loop Practices and Concepts

- Emphasis on using 'continue' over 'pass' unless deferring implementation; 'pass' is for skipping implementation, 'continue' for skipping iterations.
- Else blocks with loops are not always necessary; for and while loops in Python can use 'else' which executes after the loop unless broken.
- Nested structures in Python are introduced, with examples of nested loops (for inside for, for inside while, etc.).

Prime Number Check

- Algorithm: To check if a number is prime, iterate from 2 up to half the number (number // 2). If divisible by any number in this range, it is not prime.
- Efficient approach: Reduces iterations compared to full range iteration.
- For-else construct explained: The 'else' after a 'for' loop executes if no break occurs (i.e., when number is prime).

- Q&A clarifies the logic—using `break`, `continue`, how `else` is tied to loops, and nested `if/for` usage.
- Example: Numbers 13 and 12 are discussed as prime and non-prime, with detailed walk-throughs.

Binary Conversion with Loops

- Technique: Divide the number by 2 repeatedly, concatenating each remainder in a string to construct the binary equivalent (from lowest to highest significance).
- Loop structure: While number is not 0, continue dividing and updating the quotient for next iteration.
- Explanation of string concatenation: Remainders are converted to strings for binary representation.
- Restriction: Required to use only loops (no built-in functions like `bin()`).

Sum of Digits in a Number

- Approach: Repeatedly divide the number by 10 in a loop.
- At each iteration, the remainder (`number % 10`) is added to a running total.
- When the quotient reaches 0, the loop exits and the total sum is printed.
- Contains a step-by-step example and rationale.

Nested Loops and Break/Continue

- Several code walkthroughs illustrate how break and continue alter loop flow.
- Q&A explores: difference between breaking inner vs outer loops; how continue returns the loop to the top; role of indentation in understanding loops.
- Examples provided for tracing outputs and understanding interplay of i/j variables in nested loops.
- Advice given to manually trace code execution and run multiple examples for clarity.

Practical Coding Advice and Assignments

- Students are urged to practice at least 10–20 loop programs over the weekend for better mastery.
- Assignments are available and should be completed before new content is introduced.
- Practice platform recommendations: Assignments, W3 resource. Solutions are available but students are encouraged to run and analyze them.
- Emphasis on not simply copying solutions; learn by running, tweaking, and understanding code outcomes.
- Lists, tuples, and dictionaries (collections) will be addressed in upcoming sessions.

Q&A and Technical Issues

- Student questions address coding details, the use of break/continue, summing logic, and converting variable types.
- Technical advice on accessing assignments: Use Google Collab to open certain files from Google Drive, with step-by-step setup for students having issues.

- Students like Abhishek, Mansi, Simran, Akshans, Arif, Gunika, and Sunish participated in discussion and Q&A; Simran is the contact for assignment or batch access issues.

Conclusion

- The class concludes with study recommendations: thoroughly practice loops over the weekend for readiness.
- Lists, tuples, and further exercises will be introduced in the next class.
- Students encouraged to contact Simran or use chat/ticket support for administrative help.
- The session underscores the foundational importance of loop mastery before advancing.

Notes powered by [CraftNote](#)